

Task 3 — Cross-Browser / Cross-Platform Matrix — Scenario D — Lê Phạm Kiều Duyên, 23127184

Status: NOT YET RUN. No BrowserStack / LambdaTest trial has been used yet and no cells have been captured. This file is the coverage-floor template — fill in real cells only after real cloud-lab or real-device sessions. See `.claude/skills/cross-platform-matrix/SKILL.md` for the full method (engine-first cell selection, emulator/simulator/real-device distinction, evidence discipline).

Scope: **D1-D4**, matching Task 1B's first four screens; D5/D6 are out of scope for Task 3. Screens: **D1** (Create Support Request, user) · **D2** (My Requests + detail, user) · **D3** (Admin Support Requests list) · **D4** (Admin request detail).

Why D1-D4 satisfies the brief. §6 opens with "Tasks 1B, 2, and 3 all operate on the **same three (or more) screens** of your chosen scenario," and Task 3 itself says "Test how your **three functions/screens** render and behave." Three is the floor, not the cap: D1-D4 are four of the six screens Task 1B executed, all drawn from the Scenario D screen list in §5 (D1-D4 verbatim), so the matrix exceeds the floor while staying inside the scenario's own screen set. Every coverage claim below is therefore made **per screen, four times over**, exactly as §6 requires.

Coverage floor required (§6 Task 3), **per screen** — the brief's own words:

"Your matrix does not need every one of the 3×5×3 combinations, but it **must exercise every operating system at least once, every browser at least once, and every device class at least once, for each of the three screens**. State clearly which cells you covered and mark each **Pass / Fail**."

Required coverage values

- **OS (≥ 3):** Windows · macOS · Android **or** iOS
- **Browsers (≥ 5):** Chrome · Firefox · Safari · Edge · Opera (or Samsung Internet on mobile)
- **Device classes (3):** desktop · tablet · phone

Tooling (§6 Task 3)

The brief's order of preference, and the conditions attached to each:

Rung	Tool	Condition the brief attaches
1 (strongly preferred)	BrowserStack or LambdaTest trial	Named explicitly; obtaining trial access is the student's own responsibility (§6, §9)
2 (if the trial has expired)	Another cloud tool — Sauce Labs or CrossBrowserTesting	Same screenshot rule still applies in full

Rung	Tool	Condition the brief attaches
3 (if the trial has expired)	Real physical devices	Permitted only "provided each screenshot clearly shows the browser / OS / device name alongside the EMS URL" — on a physical device there is no cloud-lab banner, so an <code>about:</code> / device-info readout or a settings screen must be visible beside the app, or captured as a second image referenced from the same row

Record what was actually used before the first capture, so the report never has to reconstruct it:

Item	Value
Tool actually used	<i>(BrowserStack / LambdaTest / Sauce Labs / CrossBrowserTesting / physical device)</i>
Account or trial identity	<i>(the address the trial is registered to)</i>
Trial window (dates)	
Mobile OS decided (Android or iOS)	<i>(planned: Android — see <code>docs/cross_platform/00_Run_Plan.md</code> §4 item 4)</i>
Student-ID email burned into every overlay	<i>(the exact <code>MSSV@...edu.vn</code> address, identical to the one used on the §7 Google Form)</i>

Engine reminder (do not fill the matrix with one engine wearing five brand names)

Engine	Browsers in this matrix
Blink	Chrome, Edge, Opera, Samsung Internet
Gecko	Firefox
WebKit	Safari (and, by default, every iOS browser regardless of brand, unless the row states an EU/UK BrowserEngineKit exception)

Force at least one Gecko and one WebKit cell. Note the engine per row.

Screenshot requirements (§6 Task 3 + §12, mandatory)

One screenshot per cell — all 20, Pass rows included. §6: "Capture a screenshot for **every cell** in your matrix; each screenshot must overlay your username in the form **MSSV@....edu.vn** (your student-ID email)." A Fail additionally needs "a short note on the defect (overflow, overlap, broken layout, unreadable text, non-responsive control, etc.)" — recorded in the row's Note cell and in "Failures" below. §12 lists the cross-platform screenshots among the artefacts TAs verify as non-fabricable, and requires the email overlay "alongside the EMS URL and the browser/OS/device identity" — that combination is required of **every** capture, not only of the physical-device fallback where §6 words it.

Every screenshot must show, in the image itself:

1. the EMS URL,
2. the browser / OS / device identity (the lab's own banner, or an about page beside the app),
3. **your student-ID email overlay in the form `MSSV@...edu.vn`** — burn this into the image (annotation tool, lab's own overlay feature, or a visible watermark) — this is a hard anti-cheat requirement, not optional,
4. the screen in the state being claimed.

An overlay that is present but illegibly small, low-contrast, or cropped does not satisfy the rule; a caption in this markdown file or in the filename does not satisfy it either — it must be in the pixels. **A capture**

missing the overlay is not evidence: its cell reverts to `Not executed` rather than being scored on the strength of the rest of the image.

Name files: `<Screen>_<OS>_<Browser>_<Device>.png`, saved under `reports/evidence_task3/`. If the capture tool emits `.jpg` instead, keep the same stem and change the extension in the **file and the Evidence cell together** — the verifier resolves the Evidence cell against the real filename.

Environment kind — record which, per cell

The brief (§6 Task 3) requires the emulator / simulator / real-device distinction to be understood and applied; the **Environment** column in the matrix is where each cell records which one actually ran. Use exactly one of these four strings, and never leave the column blank:

`real device (local)` · `real device (cloud)` · `emulator` · `simulator`

Kind	What it proves	What it does not prove
Responsive mode (DevTools)	Layout at a resized viewport, one engine	Nothing engine- or OS-specific
Emulator (Android)	Real OS image, virtualised GPU	Real font rendering, real performance
Simulator (iOS)	Mac-native reimplementation	Real WebKit build quirks
Real device (own or cloud lab)	The actual thing	—

DevTools responsive mode is not an acceptable environment for any row in this matrix. Resizing a Windows Chrome window to phone dimensions is still Windows, still one engine, and still one device class — it cannot satisfy the phone or tablet rows, and a screenshot of it would misreport what was tested. If no real Android device or cloud real-device slot can be reached, the honest record is `emulator` in the Environment column, not a resized desktop browser labelled "phone".

Matrix

Before you press capture, on every single cell — check all five: (1) the EMS URL is visible in the image · (2) the browser / OS / device identity is visible in the image (lab banner, or device-info readout beside the app on a physical device) · (3) your `MSSV@...edu.vn` student-ID email is burned into the pixels, legibly, not covering the content being judged · (4) the screen is in the state this row claims (resting state for a `Pass`, the defective state for a `Fail`) · (5) the file is saved under `reports/evidence_task3/` under the exact filename already written in this row's Note cell. Then fill **Environment** (`real device (local)` / `real device (cloud)` / `emulator` / `simulator`), **Result** (bare `Pass` or `Fail`), and move the filename from Note into **Evidence**. Missing any of the five means the cell is not captured — leave it `Not executed` and redo it.

#	Screen	OS	Browser	Engine	Device class	Environment	Result	Evidence	Note
1	D1	Windows 11	Edge	Blink	desktop	Not executed	Not executed		capture as D1_Windows_Edge_desktop.png · Own PC, no cloud-lab minutes
2	D1	Windows 11	Firefox	Gecko	desktop	Not executed	Not executed		capture as D1_Windows_Firefox_desktop.png · Own PC; the required Gecko cell
3	D1	macOS	Safari	WebKit	desktop	Not executed	Not executed		capture as D1_macOS_Safari_desktop.png · Cloud lab; the required WebKit cell
4	D1	Android	Chrome	Blink	phone	Not executed	Not executed		capture as D1_Android_Chrome_phone.png · Cloud lab real device, or own phone
5	D1	Android	Opera	Blink	tablet	Not executed	Not executed		capture as D1_Android_Opera_tablet.png · Cloud lab real device; fills the tablet class
6	D1	iOS 17+	Safari	WebKit	phone	Not executed	Not executed		capture as D1_iOS_Safari_phone.png · Cloud lab real device , not the Xcode simulator · second WebKit environment, and WebKit-on-mobile is where this layout is most likely to break
7	D2	Windows 11	Edge	Blink	desktop	Not executed	Not executed		capture as D2_Windows_Edge_desktop.png · Own PC, no cloud-lab minutes
8	D2	Windows 11	Firefox	Gecko	desktop	Not executed	Not executed		capture as D2_Windows_Firefox_desktop.png · Own PC; the required Gecko cell
9	D2	macOS	Safari	WebKit	desktop	Not executed	Not executed		capture as D2_macOS_Safari_desktop.png · Cloud lab; the required WebKit cell
10	D2	Android	Chrome	Blink	phone	Not executed	Not executed		capture as D2_Android_Chrome_phone.png · Cloud lab real device, or own phone
11	D2	Android	Opera	Blink	tablet	Not executed	Not executed		capture as D2_Android_Opera_tablet.png · Cloud lab real device; fills the tablet class
12	D2	iOS 17+	Safari	WebKit	phone	Not executed	Not executed		capture as D2_iOS_Safari_phone.png · Cloud lab real device , not the Xcode simulator · second WebKit environment, and WebKit-on-mobile is where this layout is most likely to break
13	D3	Windows 11	Edge	Blink	desktop	Not executed	Not executed		capture as D3_Windows_Edge_desktop.png · Own PC, no cloud-lab minutes
14	D3	Windows 11	Firefox	Gecko	desktop	Not executed	Not executed		capture as D3_Windows_Firefox_desktop.png · Own PC; the required Gecko cell
15	D3	macOS	Safari	WebKit	desktop	Not executed	Not executed		capture as D3_macOS_Safari_desktop.png · Cloud lab; the required WebKit cell
16	D3	Android	Chrome	Blink	phone	Not executed	Not executed		capture as D3_Android_Chrome_phone.png · Cloud lab real device, or own phone
17	D3	Android	Opera	Blink	tablet	Not executed	Not executed		capture as D3_Android_Opera_tablet.png · Cloud lab real device; fills the tablet class
18	D3	iOS 17+	Safari	WebKit	phone	Not executed	Not executed		capture as D3_iOS_Safari_phone.png · Cloud lab real device , not the Xcode simulator · second WebKit environment, and WebKit-on-mobile is where this layout is most likely to break
19	D4	Windows 11	Edge	Blink	desktop	Not executed	Not executed		capture as D4_Windows_Edge_desktop.png · Own PC, no cloud-lab minutes
20	D4	Windows 11	Firefox	Gecko	desktop	Not executed	Not executed		capture as D4_Windows_Firefox_desktop.png · Own PC; the required Gecko cell
21	D4	macOS	Safari	WebKit	desktop	Not executed	Not executed		capture as D4_macOS_Safari_desktop.png · Cloud lab; the required WebKit cell
22	D4	Android	Chrome	Blink	phone	Not executed	Not executed		capture as D4_Android_Chrome_phone.png · Cloud lab real device, or own phone

#	Screen	OS	Browser	Engine	Device class	Environment	Result	Evidence	Note
23	D4	Android	Opera	Blink	tablet	Not executed	Not executed		capture as <code>D4_Android_Opera_tablet.png</code> · Cloud lab real device; fills the tablet class
24	D4	iOS 17+	Safari	WebKit	phone	Not executed	Not executed		capture as <code>D4_iOS_Safari_phone.png</code> · Cloud lab real device , not the Xcode simulator · second WebKit environment, and WebKit-on-mobile is where this layout is most likely to break

Row plan (covering array, not full cross-product). The brief states the coverage floor **per screen**, and no cell can carry more than one browser, so a screen cannot cover 5 required browsers in fewer than 5 rows: the floor is `max(3 OS, 5 browsers, 3 device classes) = 5` cells per screen, hence a floor of **20 cells across D1-D4. This matrix runs 24, one above the floor per screen.** The sixth row is **iOS + Safari + phone on a real device**, added deliberately: at the bare floor, three of the five required brands (Chrome, Edge, Opera) are Blink, so two of the three engines would be exercised exactly once per screen and mobile WebKit — the combination most likely to break this layout — not at all. The extra row buys a fourth OS, a second WebKit environment, and the real-device-versus-simulator distinction the brief asks to be understood. The same 6-tuple is applied to each screen so the coverage argument is identical on every one of them, and so that one cloud-lab session can capture all four screens before it is closed. Checked against the floor: OS {Windows, macOS, Android} = 3/3 · browsers {Edge, Firefox, Safari, Chrome, Opera} = 5/5 · device classes {desktop, phone, tablet} = 3/3 · engines {Blink, Gecko, WebKit} = 3/3, so the matrix is not one engine wearing five brand names.

Three of the five brands are Blink (Chrome, Edge, Opera) — only Firefox (Gecko) and Safari (WebKit) add real engine diversity. Rows beyond this floor, if the trial allows them, should go to WebKit at a narrow viewport rather than to another Blink brand.

Each row's capture filename is pre-assigned in its Note cell, so a screenshot can be named at the moment it is taken and never reconciled afterwards. When a cell is captured, move that filename into the **Evidence** column and replace the Note with the defect description (Fails only) — the Evidence column stays empty until a real image exists, so `matrix_coverage.py` only ever complains about a reference that is genuinely wrong. If the mobile OS decision changes from Android to iOS (see `docs/cross_platform/00_Run_Plan.md` §4 item 4), rewrite rows 4-5 of each screen **and** their filenames together, and note the WebKit-by-default rule: on iOS every browser is WebKit regardless of brand unless the row documents an EU/UK BrowserEngineKit exception.

Run order that collapses these 24 cells into **4 metered cloud-lab launches** (the local Windows rows are free): `docs/cross_platform/00_Run_Plan.md` §6.

Coverage achieved (fill in after running)

Screen	OS covered	Browsers covered	Device classes covered	Engines covered	Missing
D1					
D2					
D3					
D4					

Verify with (run from `HW03/`):

```
python .claude/skills/cross-platform-matrix/scripts/matrix_coverage.py
docs/04_Task3_Cross_Platform_Matrix.md \
  --os "Windows,macOS,Android" \
  --browsers "Chrome,Firefox,Safari,Edge,Opera" \
  --devices "desktop,tablet,phone" \
  --evidence-root reports/evidence_task3/
```

`--os` must name **exactly the three operating systems the rows actually use** — every value listed is treated as required, so adding `iOS` "for safety" makes the script report `OS not covered: iOS` on all four screens and exit 1 even though the matrix is correct. If the mobile-OS decision changes from Android to iOS, change the rows **and** this command together. A clean run prints `3/3, 5/5, 3/3` per screen with no `MISSING`, at least two engines per screen, and ends `OK -- coverage floor met on every screen,`

evidence resolves with exit code 0. It passes clean today, unrun — so any later failure is a real regression, not scaffolding noise.

Failures (grouped by classification)

§6 Task 3: "Attach screenshots for any rendering/layout **Fail** with a short note on the defect (overflow, overlap, broken layout, unreadable text, non-responsive control, etc.)." One row here per **Fail** cell in the matrix above — the same screenshot, plus the note.

Use the fixed vocabulary: overflow · overlap · clipping · unreadable · unresponsive-control · missing-asset · feature-unsupported · broken-layout . Reproduce any Fail once (ideally a fresh session) before logging it, and cross-check the image itself — cloud labs can serve a stale screenshot mid-load. Separate a genuine incompatibility from a legitimate platform difference (native date-input locale format, platform scrollbars, system font substitution, the mobile URL bar changing viewport height) — the latter goes in the next section, not here.

Cell #	Screen	OS / Browser / Device	Classification	Short note on the defect	Evidence	Findings-log ID

(None recorded yet — no cell has been captured.)

Handoff to §7 (Bug & Usability Findings)

Every genuine incompatibility found here must be reported **twice**, per §7 of the brief:

1. **Google Form** — <https://forms.gle/CJQFQCAXcsDbXDMM9>, submitted from the same student-ID email that is burned into the screenshot overlays, so the submission is attributable.
2. **docs/05_Bug_Usability_Findings_Log.md** — typed Bug , with the matrix cell (screen, OS, browser, device class, environment kind) as its reproduction context and the cell's screenshot as its evidence reference. Record the form-submission timestamp in the log's own column; §7 warns the aggregated file and the form submissions must be consistent and the TA may cross-check counts.

New IDs start at **D-020** (D-001...D-019 are taken by Task 1B). **D-013, D-014 and D-018 are retired and must never be reused.** Task 2 also draws from D-020 onward — allocate an ID only at the moment a finding is actually written, never reserve a block in advance, and record the ID back into the "Findings-log ID" column above.

Platform differences observed (not defects)

Record legitimate platform behaviour here so a reviewer sees it was considered, not missed — e.g. a native From date / To date input on the Support Requests filter panel (D3) rendering in the browser's own locale format is expected, not a bug (see docs/01_Task1A_Shared_GUI_Checklist.md IA02-11). (None recorded yet.)